

Resilient Secure Communication for Multi-UAV Systems under Network Attacks

Jasmin Steinbrecher

Department of Aerospace Engineering, Georgia Institute of Technology

jsteinbrecher3@gatech.edu

December 10, 2025

Abstract

Consensus-based coordination is a key enabler for multi-Unmanned Aerial Vehicle (UAV) swarms in applications such as search-and-rescue, environmental monitoring, and autonomous delivery. These systems rely on local state exchanges and standardized communication protocols, which, while efficient, introduce critical vulnerabilities to operational faults and adversarial interference. Ensuring safe and reliable swarm behavior therefore requires a careful evaluation of both consensus mechanisms and underlying network protocols.

This paper systematically evaluates the resilience of UAV swarms executing the Weighted Mean-Subsequence Reduced (W-MSR) consensus algorithm and communicating over the widely used Micro Aerial Vehicle Link (MAVLink) protocol. The developed simulation shows how network sparsity, filtering parameters, and malicious state injections impact consensus convergence and swarm cohesion. The results reveal that sparse networks are prone to fragmentation even under modest adversarial influence, while dense networks, although robust to malicious state injections, remain vulnerable to high-rate communication and resource-exhaustion attacks. These findings highlight the need to jointly consider consensus design and protocol-level security to achieve safe and resilient multi-UAV operations.

1 Introduction

Drone swarms that coordinate to converge on an agreed location support high-stakes applications such as search and rescue, environmental monitoring, and autonomous aid delivery (Ficco et al., 2022). These scenarios rely on accurate and timely exchange of positional and state information among the UAVs, forming a mobile network whose connectivity and integrity directly influence mission success (Pasqualetti et al., 2012).

Distributed coordination, however, introduces significant network security challenges (Ficco et al., 2022). Each drone makes decisions based on information received

from neighbors, meaning that even a single faulty or malicious drone can compromise the swarm’s global behavior (Ficco et al., 2022; Olfati-Saber et al., 2007; Pasqualetti et al., 2012). Misreported positions or states can mislead consensus mechanisms, degrade network connectivity, and potentially fragment the swarm, disrupting mission objectives and introducing safety risks (Du et al., 2024).

Resilient consensus algorithms, such as the W-MSR approach, are designed to mitigate the impact of faulty or adversarial nodes by filtering extreme neighbor values before state updates (Olfati-Saber et al., 2007). However, their effectiveness to drive consensus behavior in UAV swarms depends on network topology, connectivity, and the underlying communication protocol. In practice, the widely used MAVLink protocol was designed for reliability and interoperability rather than security, leaving swarms exposed to message injection, spoofing, and resource-exhausting attacks (Ficco et al., 2022).

This paper systematically analyzes the resilience of consensus-based UAV swarms to adversarial attacks, using a custom Python simulator to model swarm behavior as drones attempt to reach consensus. The study focuses on W-MSR performance under varying network topologies and attacker behaviors. Observed vulnerabilities are mapped to known weaknesses in the MAVLink protocol, demonstrating how protocol-level flaws can amplify risks in swarm coordination and undermine mitigation strategies such as the W-MSR algorithm. The simulation results provide actionable insights into designing safer and more reliable multi-UAV systems in real-world environments.

2 Related Work

Consensus algorithms play a central role in coordinating distributed multi-agent systems by enabling agents to reach consensus on shared variables through local information exchange. They are generally defined as “an interaction rule that specifies the information exchange between an agent and all of its neighbors on the network” (Olfati-Saber et al., 2007). Classical consensus schemes

operate under the assumption that all agents are cooperative, follow the prescribed update rule, and communicate accurate information. Under these conditions, with each agent updating its state as a weighted combination of its neighbors, a network-wide agreement can be reached in connected graphs (Pasqualetti et al., 2012; Saldaña et al., 2017).

However, real-world systems, such as UAV swarms, rarely operate in ideal conditions. Communication faults, hardware failures, and adversarial interference can introduce non-cooperative agents whose transmitted states no longer reflect their true value (Saldaña et al., 2017). Such agents may be defective, broadcasting false information due to unintentional failure, or malicious, coordinating to disrupt the system and corrupt the consensus process (Pasqualetti et al., 2012). Since consensus protocols rely heavily on local interactions, even a small number of compromised nodes can cause divergence or fore the network toward incorrect values (Nakamura et al., 2021).

To address these vulnerabilities, a class of resilient consensus algorithms known collectively as mean subsequence reduced (MSR) algorithms has been developed (Nakamura et al., 2021). In this paper, specifically, the W-MSR algorithm was used. It operates by filtering out extreme or suspicious neighbor values before applying the consensus update. By rejecting the F largest and smallest received values, the algorithm can guarantee convergence even when up to F neighbors behave maliciously or unpredictably (Nakamura et al., 2021; Saldaña et al., 2017). These methods require only local information and do not depend on global knowledge of the network structure, making them suitable for large and time-varying systems (Saldaña et al., 2017).

While resilient consensus algorithms, such as W-MSR, provide the theoretical foundation for achieving agreement in a network, their successful implementation in UAV swarms depends on the underlying communication infrastructure. In real-world deployments, state updates and messages must be exchanged over a standardized protocol to ensure interoperability and reliability. For this reason, many UAV systems operating with ArduPilot or PX4 autopilot frameworks rely on the lightweight MAVLink protocol, which defines a structured message format for exchanging telemetry, commands, and state information between drones or Ground Control Stations (GCSs) (Ficco et al., 2022; Koubâa et al., 2019). The protocol supports two main versions: MAVLink 1.0, released in 2009, and MAVLink 2.0, released in 2017, which is backward-compatible with MAVLink 1.0 and introduces optional message signing for authentication.

Despite its widespread adoption, MAVLink was primarily designed with safety and availability in mind, rather than

security (Ficco et al., 2022). Several vulnerabilities in both MAVLink 1.0 and 2.0 can compromise the integrity, confidentiality, and authenticity of communications:

- **Optional or weak signatures:** MAVLink 2.0 allows message signing, but it is optional, and many implementations do not enable it by default. The signature relies on the first 48 bits of a SHA-256 hash, which is susceptible to forgery and collision attacks (Ficco et al., 2022).
- **Exposure of secret keys:** During the setup signing procedure, secret keys may be transmitted over unencrypted channels if specifications are not strictly followed, enabling attackers to capture keys and inject malicious messages (Ficco et al., 2022).
- **Timestamp vulnerabilities:** MAVLink messages rely on timestamps for ordering and replay prevention, but mismanagement of timestamps can allow replay attacks, Denial-of-Service (DOS), or message injection (Ficco et al., 2022).
- **Lack of encryption:** By default, MAVLink messages are transmitted in plaintext, exposing all telemetry and control data to eavesdropping or spoofing by adversaries within wireless range (Ficco et al., 2022).

These vulnerabilities have direct implications for resilient consensus algorithms like W-MSR, as they can be exploited to introduce adversarial nodes into the network, potentially making the filtering algorithm either protective or detrimental to overall swarm behavior. Recent research has proposed several mitigations, including enforcing mandatory message signing, integrating lightweight cryptography such as in MAVSec and MAVShield, and improving key management practices (Allouch et al., 2019; Dixit et al., 2025). In this work, the security limitations of MAVLink are incorporated into the threat model to guide the design and evaluation of W-MSR consensus in real-world UAV swarms. Specifically, this paper investigates how W-MSR’s behavior in UAV swarms can generate security vulnerabilities and analyzes how these vulnerabilities align with known weaknesses in the MAVLink protocol.

3 Threat Model

This paper considers adversaries that target the integrity of inter-UAV communication within a swarm coordinated through consensus-based control. The focus is on attacks exploiting weaknesses in the MAVLink protocol and limitations in swarm connectivity.

The system consists of a swarm of N UAVs communicating over wireless links using MAVLink. Each agent ex-

changes state information (position for simplicity) with a fixed set of neighbors determined by a k -nearest-neighbor topology. Each UAV can run the W-MSR algorithm and update its state based on locally received messages.

Communication occurs over User Datagram Protocol (UDP), following port-increment patterns commonly used by MAVLink-based autopilot stacks (discussed in Section 6). Furthermore, no transport-layer encryption or authentication is assumed. This reflects default MAVLink v1 behavior and the widespread use of unsigned MAVLink v2, where message authentication is typically disabled due to key-distribution overhead and incompatibility with GCSs (Ficco et al., 2022). MAVLink v1 remains available via a legacy compatibility channel and is used when connections over MAVLink v2 fail (MAVLink Project, 2019).

3.1 Attacker Goals

The adversary aims to degrade or manipulate swarm coordination by influencing the consensus process. Their specific objectives are:

1. Prevent consensus by introducing disagreement or fragmentation.
2. Steer subgroups of the swarm toward attacker-defined states.
3. Exploit graph topology (low or high connectivity) to maximize the effect of malicious nodes.
4. Remain stealthy by blending into network traffic or mimicking legitimate state broadcasts.

The attacker does not directly modify the flight controller or autopilot firmware itself. Instead, the attacker manipulates communicated position states and may induce operational disruption by overwhelming the communication channel.

3.2 Attacker Capabilities and Limitations

Since MAVLink v1 and unsigned MAVLink v2 lacks cryptographic authentication, the attacker can inject forged messages such as `GLOBAL_POSITION_INT` or `LOCAL_POSITION_NED`. A simple script using `pymavlink` can discover active UDP ports, initiate MAVLink sessions without credentials, and transmit arbitrary spoofed state updates. Additionally, the attacker can capture MAVLink packets via Wi-Fi sniffing or a man-in-the-middle proxy and replay stale messages (Ficco et al., 2022).

However, the adversary cannot modify physical flight dynamics, alter autopilot firmware, jam radio channels at the physical layer, or compromise onboard sensors such as GPS. Their influence is restricted to the network layer

through MAVLink message manipulation and the resulting processing load on the receiver. These constraints reflect realistic adversaries in civilian or prototyping UAV systems, where remote code execution or hardware compromise is significantly more difficult than network-level spoofing.

3.3 Defender Assumptions

The UAV swarm relies on distributed neighbor-based consensus, with a variable number of k -nearest-neighbor links and optional W-MSR filtering with parameter f . No global controller or supervisory node is assumed, which is consistent with decentralized swarm architectures.

4 Approach

To identify the security threats that most critically affect consensus-based coordination in UAV swarms, a simulation-driven methodology was adopted. A multi-agent consensus system was modeled in Python and evaluated under both nominal and adversarial conditions. The resulting failure modes were analyzed to determine how network-layer attacks influence swarm behavior. These empirical observations were then mapped to known weaknesses in the MAVLink communication protocol (Section 2) to assess which cyber threats are realistically exploitable in real deployments.

4.1 UAV Swarm Simulation

A custom Python simulator was developed to model a swarm of N UAVs executing a W-MSR consensus update at each time step. The simulator shows key parameters, including swarm size, number of simulation steps, consensus update step size, the W-MSR filtering parameter f , neighbor count k , number of adversarial agents, and attacker position deviation strength.

The core configuration is:

```

1   N = 20
2   T = 300
3   step = 0.2
4   f_list = [0,1,2]
5   k_list = [5,10,15]
6   attacker_count_list = [1,2,3]
7   attack_offset_list = [0.5,1.0]
```

For simulation and reproducibility purposes, the 20 modeled drones are distributed evenly in a circle around the origin. Their goal is to agree on flying to the origin. In practice, UAV swarms will have a limited number of UAVs available and the UAV swarm platforms have a limit to how many drones they can support (Alqudsi & Makaraci, 2025; Friess et al., 2024; Stolfi & Danoy, 2025). To reflect hardware and bandwidth limits, $N = 20$ was chosen as a realistic deployment scale. With $k \in$

$\{5, 10, 15\}$ the swarm transitions across low, medium, and high connectivity regimes, allowing an evaluation of how W-MSR performs as network structure changes.

4.2 Adversary Modeling

Adversarial drones are modeled as nodes that intentionally broadcast falsified position states to mislead the consensus process. This behavior is represented in standard categories of network-layer attacks, including false data injection, message omission or packet dropping, message delay manipulation, or spoofing (Stallings, 2011). It is assumed that all of them essentially result in a falsified position report by the agent.

For each simulation run, adversarial behaviors are systematically varied, and metrics such as consensus error, convergence time, and swarm cohesion are recorded. Consensus is defined as the first time step where the mean distance of all agents to the swarm centroid falls below 0.1 in simulation distance units. Convergence time denotes the number of simulation steps required to reach this consensus condition. Swarm cohesion is assessed qualitatively through trajectory visualizations of the 20-agent swarm, which illustrate clustering, fragmentation, and overall spatial organization under different attack scenarios. Based on these results, disruptions causing the most severe operational instability are identified. Finally, these observed vulnerabilities are then mapped to MAVLink protocol properties to determine which cyber threats are realistically exploitable.

4.3 W-MSR Filtering

The W-MSR algorithm provides resilience by removing up to f of the highest and lowest neighbor values before performing the consensus update. Formally, each agent i receives a set of neighbor states $N_i(t)$ and constructs a filtered set $N_i^*(t)$ by removing the f largest and f smallest values along each dimension. The updated state is then determined by:

$$x_i(t+1) = w_{ii}x_i(t) + \sum_{j \in N_i^*(t)} w_{ij}x_j(t)$$

where the weights w_{ij} are non-negative and sum up to 1 (Saldaña et al., 2017). In this work, a uniform weighting scheme is used, reflecting the absence of leader agents or hierarchical control.

In practice, applying W-MSR reduces the effective connectivity of the swarm, since valid neighbor states may be discarded whenever the number of extreme values exceeds the filtering threshold. This creates a trade-off between resilience and cohesion: higher f -values improve robustness to adversaries but increase the risk of fragmenting the swarm or slowing convergence. The W-MSR

implementation used in the simulator is shown below:

```

1 filtered = []
2 for d in range(dim):
3     vals = neighbor_pos[:, d]
4     vals_sorted = np.sort(vals)
5     if f == 0 or len(vals_sorted) <= 2*f
6         :
7         trimmed = vals_sorted
8     else:
9         trimmed = vals_sorted[f:-f]
```

5 Results

The simulation experiments were performed using a swarm of 20 drones, a scale representative of small-to medium-sized research and civilian (Friess et al., 2024; Stolfi & Danoy, 2025). This swarm size is sufficiently large to exhibit emergent collective behavior while remaining tractable for controlled experimentation and analysis of resilience under adversarial conditions.

Each swarm instance operated under a k -nearest neighbor communication topology with $k \in \{5, 10, 15\}$, representing sparse, moderate, and dense connectivity regimes. Adversarial influence was modeled by introducing up to three attacker drones, each injecting a false position with an offset of either 0.5 or 1.0 simulation units. All runs executed for 300 time steps, providing adequate duration to observe convergence, divergence, and instability effects. Simulations were carried out on a workstation with an AMD Ryzen 7 5800H CPU, 16 GB RAM, running Ubuntu 20.04.

5.1 Baseline Behavior

Figure 1 illustrates the swarm trajectories under benign conditions without W-MSR filtering. For sparse connectivity ($k = 5$), the swarm does not reach consensus within the 300-step window. However, visual inspection of Figure 1 shows that agents still cluster around the origin, indicating that the system remains cohesive despite the lack of formal convergence. The consensus error plot in Figure 2 confirms this, showing a final error of 0.16 for $k = 5$. Extending the simulation window would likely allow eventual convergence.

For $k = 10$ and $k = 15$, the swarm converges within the allotted time, as shown in the consensus error plot (Figure 2) and the trajectory plots (Figure 1). As expected, higher connectivity results in faster and more stable convergence. Final consensus errors are on the order of 7.53×10^{-13} for $k = 10$ and 7.13×10^{-18} for $k = 15$, indicating near-perfect alignment.

Applying the W-MSR filtering significantly alters swarm behavior, particularly in low-connectivity scenarios. With $k = 5$, even a single filtering level ($f = 1$) prevents the swarm from reaching consensus. The system begins to

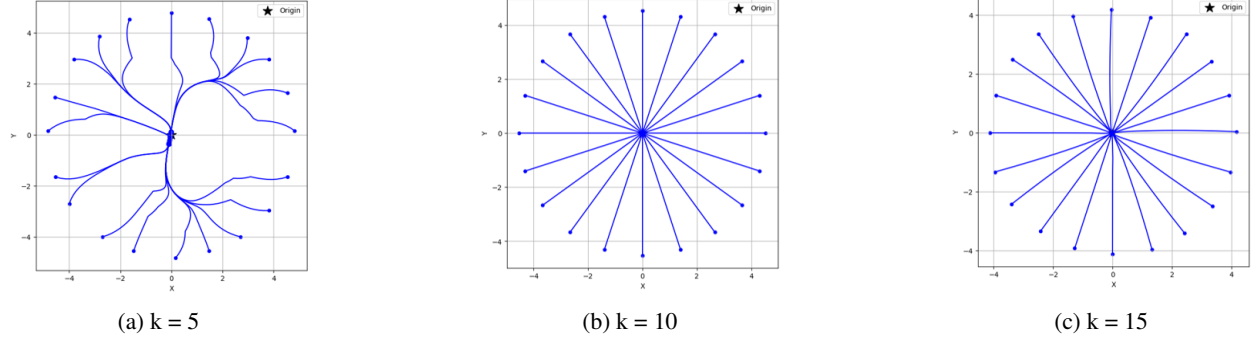


Figure 1: Simulation results for different k -nearest neighbor topologies, without attackers and without W-MSR Filtering.

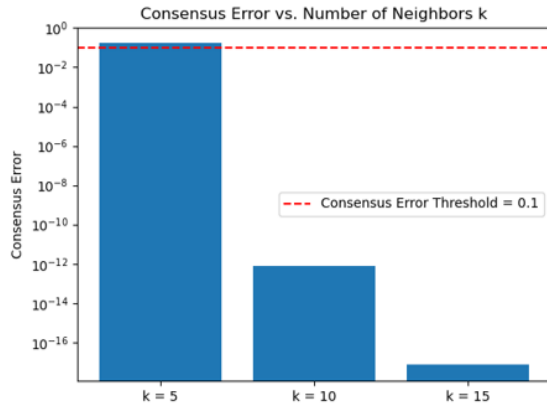


Figure 2: Consensus error plot corresponding to the trajectories shown in Figure 1.

form clusters as shown in the trajectory plots in Figure 3. This occurs because W-MSR removes the f largest and f smallest neighbor values at each iteration, reducing the effective in-degree of the network. Under $k = 5$ and $f = 2$, four of five neighbor observations are discarded, leaving the agent with only one effective input. Although this satisfies the nominal requirement $f < k/2$, it violates the $(2f + 1)$ -robustness condition needed for resilient consensus (Saldaña et al., 2017). As illustrated in Figure 4, the consensus errors rise sharply from 0.1665 ($f = 0$) to 3.84197 ($f = 1$) and 4.58997 ($f = 2$), confirming the loss of global agreement.

For higher connectivity values ($k = 10$ and $k = 15$), the swarm maintains consensus even under filtering. In these cases, each agent retains enough valid neighbor values after trimming to preserve the connectivity properties required for W-MSR to operate correctly. Convergence speed remains correlated with network density, with $k = 15$ consistently reaching consensus fastest.

Overall, the baseline results show that W-MSR filtering

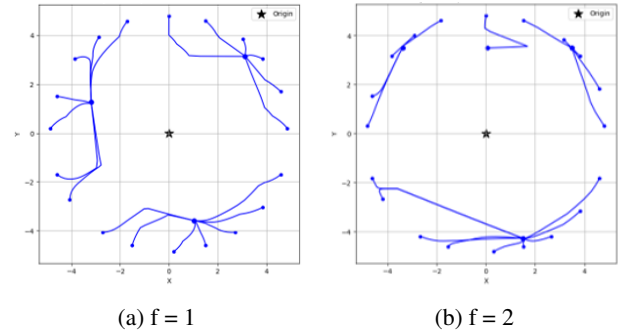


Figure 3: Trajectory plots for $k = 5$ showing the effect of different filter values f , with no attackers present.

imposes a non-trivial connectivity requirement: while it improves adversarial resilience, it degrades performance or even prevents consensus in sparsely connected networks. These observations provide a necessary reference for evaluating adversarial disruptions in the next section.

5.2 Attack Behavior

To assess the resilience of the swarm under adversarial conditions, simulation experiments were performed using the same 20-UAV configuration as in the baseline analysis. Adversaries were modeled as drones that inject bounded random perturbations into their transmitted positions at each timestep. The magnitude of these perturbations was controlled via the parameter $\text{attack_offset} \in \{0.5, 1.0\}$, and scenarios with up to three attackers were evaluated. Attacker indices were selected deterministically (first n agents) to allow reproducibility.

Simulation runs were conducted for all combinations of the network connectivity parameter $k \in \{5, 10, 15\}$ and the W-MSR filtering parameter $f \in \{0, 1, 2\}$. Each run lasted 300 time steps, and the baseline case was retained for reference.

Figure 5 shows representative swarm trajectories for $k =$

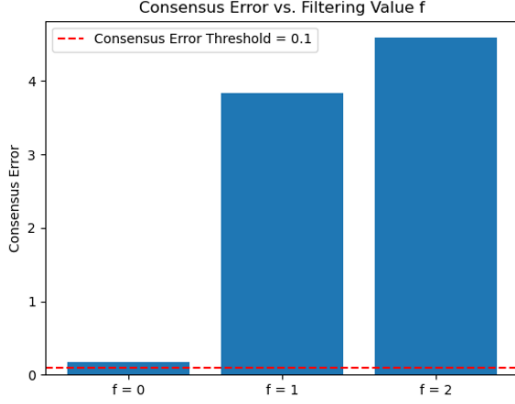


Figure 4: Consensus error plot corresponding to the baseline case ($k = 5$, $f = 0$) and the trajectories shown in Figure 5.

5 under a single attacker and varying filter strengths. Swarm behavior in this regime is highly sensitive to W-MSR filtering. For $f = 0$, the swarm remains cohesive and exhibits similar behavior to baseline. For $f = 1$, consensus progressively degrades and the swarm drifts away from the origin. For $f = 2$, the swarm consistently fragments into multiple disconnected clusters. This outcome is expected: when k is small, removing up to f extreme neighbor values on both ends reduces the effective in-degree below the $(2f + 1)$ -robustness condition required for W-MSR convergence (Saldaña et al., 2017). Thus, even a single attacker is sufficient to destabilize sparsely connected swarms because the network lacks the connectivity required for W-MSR to filter adversarial input.

For higher connectivity levels ($k = 10$ and $k = 15$), the swarm demonstrates strong resilience to adversarial influence. Consensus is achieved in all attack scenarios, including the most adversarial configuration (three attackers, $f = 2$, attack offset 1.0). The trajectory results for this worst-case scenario are shown in Figure 6, and the corresponding consensus errors in Figure 7. Under $k = 5$, the swarm fails to converge, reaching a final consensus error of 4.59. In contrast, for $k = 10$ and $k = 15$, the swarm maintains full connectivity and converges to a common position with final errors on the order of 6.6×10^{-9} and 1.7×10^{-17} respectively. As expected, these consensus errors are higher than the baseline consensus errors described in Section 5.1. This demonstrates that increasing neighborhood degree dramatically improves the robustness to bounded adversarial perturbations.

Across all evaluated scenarios, connectivity emerges as the dominant factor governing robustness. W-MSR filtering suppresses adversarial influence effectively only when the network meets its in-degree requirements; otherwise, filtering itself can destabilize the swarm. In sparse net-

works ($k = 5$), even modest filtering leads to fragmentation, while moderate or dense networks ($k \geq 10$) retain stable consensus even under attacks. The operational implications for MAVLink-enabled multi-agent systems are examined in Section 6.

6 Discussion

The simulation results in Section 5 highlight that network connectivity is the primary factor determining consensus in UAV swarms. While W-MSR filtering can mitigate some adversarial influence, it cannot compensate for low connectivity.

6.1 Vulnerabilities of sparse swarms

The results show that low-connectivity swarms ($k = 5$) are fundamentally vulnerable, independent of whether W-MSR filtering is enabled. While W-MSR is theoretically resilient to up to f misbehaving neighbors per node, the algorithm fails in sparse networks because trimming removes too many neighbor values, reducing the effective in-degree below the threshold required for consensus (Nakamura et al., 2021; Saldaña et al., 2017). In practice, this means that low-connectivity swarms cannot rely on W-MSR as a viable defense mechanism.

Even if W-MSR were adapted, for example by only activating when outlier positions exceed a certain threshold, the swarm remains vulnerable. In the always-on case, attackers may inject arbitrary position values with no filtering resistance. In the threshold-triggered case, the swarm may preserve connectivity slightly longer, but the results show that even small perturbations in low-degree networks prevent convergence. Thus, the attack surface remains open in both variants.

These vulnerabilities map directly to known weaknesses in MAVLink, which were discussed in Section 2. Any entity capable of sending UDP packets to the swarm’s communication channel can:

1. Impersonate another drone since identification fields are unauthenticated and consequently send spoofed position messages
2. Replay messages as MAVLink does not include timestamps or sequence numbers preventing replay attacks.

These attacks are comparatively easy to deploy since UAV swarms rely on predictable communication patterns. ArduPilot SITL or PX4 SITL use standardized port assignments to manage the connections between UAVs. UDP ports typically begin at 14550 and increment by 10, while TCP ports increment by 100. This regularity makes enumeration trivial.

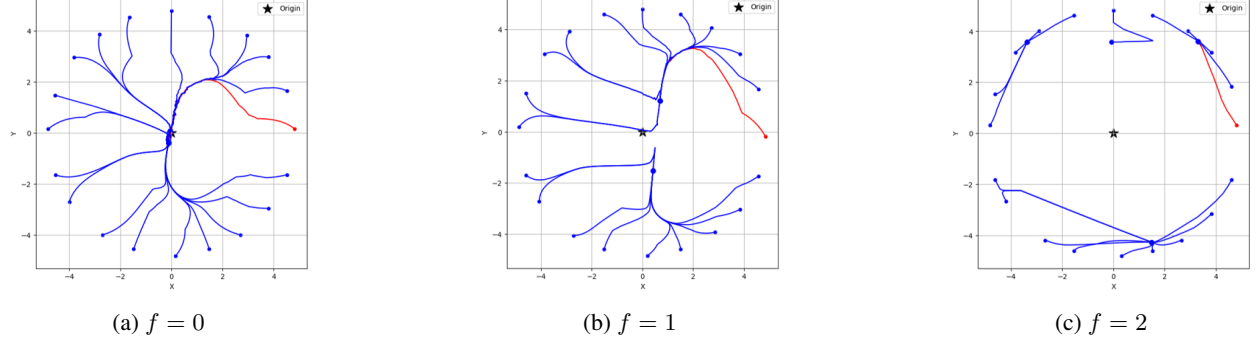


Figure 5: Swarm trajectories for $k = 5$ with a single adversarial agent, shown for different W-MSR filter parameters $f \in \{0, 1, 2\}$.

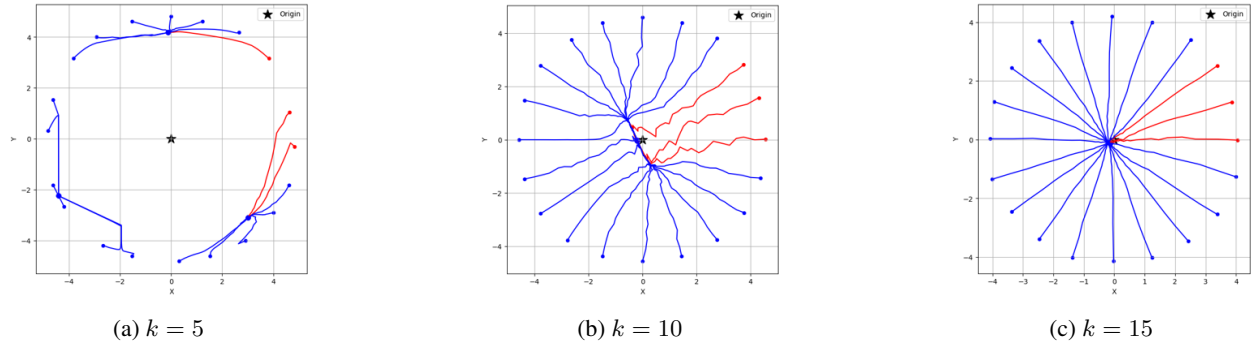


Figure 6: Swarm trajectories for $k \in \{5, 10, 15\}$ under three attackers, with $f = 2$ and an attacker offset of 1.0, representing the most adversarial conditions for the swarm.

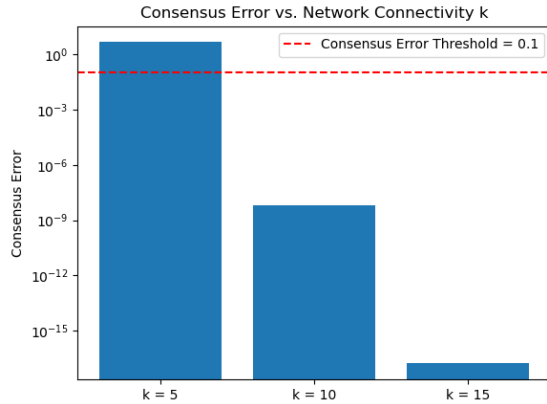


Figure 7: Consensus error plot corresponding to the trajectories shown in Figure 6

Using pymavlink, an attacker can connect to any drone whose port is reachable:

```
1 DRONE_PORTS = [14550, 14560, 14570,
2                 14580]
3 drones = []
4 for port in DRONE_PORTS:
5     mav = mavutil.mavlink_connection(f'
```

```
udp:127.0.0.1:{port}')
6 mav.wait_heartbeat()
7 print(f"Connected to drone on port {
8       port}")
9 drones.append(mav)
```

Once connected, an attacker can transmit arbitrary MAVLink messages preventing the swarm from converging.

In the simulation, attackers were chosen to be the first n drones to observe the overall behavior of the swarm. In a real swarm, however, an intelligent adversary would place attackers across different regions of the communication graph to exploit its fragility and fragmentation. Once the swarm inevitably fragments, as shown in Section 5, each subgroup evolves independently. This enables coordinated hijacking or redirection of separated clusters.

Once an attacker is connected to a drone, the only way to remove them is to terminate the SITL instance. This kills the autopilot process, making the drone transition to its configured fail-safe behavior, such as hover, land, return-to-launch or crash in the worst-case.

In practice, lower connectivity may be chosen in drone swarms to reduce communication overhead, conserve en-

ergy, or limit bandwidth usage. However, the results indicate that sparse networks are highly susceptible to fragmentation and disruption.

6.2 Vulnerabilities of denser swarms

High-connectivity swarms ($k \geq 10$) demonstrated strong resilience to adversarial consensus manipulation in the simulation. Dense networks have sufficient redundancy to determine their position, preventing fragmentation even when several nodes are compromised. However, this robustness introduces a different attack surface at the network and system layers. High-connectivity swarms are not inherently secure. Instead, they trade vulnerability to consensus disruption for vulnerability to resource exhaustion, link saturation, and protocol-level overload attacks.

The nature of highly connected UAV swarms requires numerous simultaneous communication links. Each drone may exchange state information with dozens of peers at high frequency. The precise update rate is implementation-dependent and varies with the mission requirements, communication bandwidth, and experimental setup (Alqudsi & Makaraci, 2025). MAVLink listeners typically accept all messages arriving on a port, providing no built-in rate limiting at the protocol level. An adversary can therefore transmit a high-rate stream of arbitrary MAVLink frames, exceeding the receiver’s packet processing capacity. Autopilots, such as ArduPilot, execute their flight-control logic within a software-scheduled task framework that allocates bounded time budgets to control-loop tasks, sensor updates and other auxiliary functions. Although ArduPilot employs priority-based scheduling, excessive MAVLink traffic consumes available processing time and may lead to increased control latency, skipped sensor-fusion updates, Extended Kalman Filter (EKF) divergence or autopilot crashes in severe cases. In a high-connectivity swarm, the baseline communication load is already significant, so an attacker needs only a modest additional message rate to push the system into overload. This constitutes an effective DOS attack on the flight controller.

Even in MAVLink v2, which supports signing for authentication, does not prevent high-rate attacks. Signing ensures message authenticity but does not impose any rate-limiting on incoming messages. Consequently, a malicious agent can still inject large volumes of traffic. While the autopilot may discard unsigned or invalid messages after parsing headers, processing these headers consumes CPU resources.

6.3 Limitations of the Simulation

Several simplifications in the current simulation framework should be acknowledged. First, the swarm operates in a two-dimensional plane, whereas real UAVs move in three dimensions, potentially introducing additional de-

grees of freedom and failure modes. Second, communication delays and packet loss were not modeled since they were assumed to result in an additional misrepresentation of the drone’s position. Third, attacker positions were assigned deterministically rather than randomly or strategically, which may underestimate the potential for disruption in real-world scenarios. Finally, both the W-MSR parameters and the adversary model are simplified relative to actual UAV behavior, and the interactions between network dynamics and vehicle control loops are not captured in full detail. These simplifications should be considered when interpreting the results.

6.4 Future Work

Future work should extend this analysis to high-fidelity Software-in-the-Loop (SITL) simulations, incorporating realistic UAV dynamics and MAVLink-based communication protocols. Such experiments would enable systematic evaluation of encrypted versus unencrypted channels, variable network connectivity, and more complex attacker placement strategies. Validating the observed vulnerabilities under near-realistic operational conditions will provide actionable guidance for developers of UAV swarms, informing the design of resilient consensus algorithms, secure communication strategies, and mission planning practices for safe and robust swarm operation.

7 Conclusion

This paper systematically analyzed consensus-based UAV swarms operating over the widely used MAVLink protocol to identify potential security and operational vulnerabilities. Even when employing W-MSR filtering, the simulations revealed that sparse networks are highly susceptible to fragmentation and consensus failure, while dense networks, though resilient to malicious state injections, are vulnerable to high-rate communication and resource-exhaustion attacks. An adversary can exploit predictable communication patterns, unauthenticated MAVLink messages, and high-connectivity traffic loads to disrupt swarm behavior. These findings highlight that both consensus-level mechanisms and protocol-level design must be carefully considered to ensure safe and reliable swarm operations. Future work will extend these results to high-fidelity simulations and real-world experiments to further validate vulnerabilities and mitigation strategies.

References

- Allouch, A., Cheikhrouhou, O., Koubâa, A., Khalgui, M., & Abbes, T. (2019). Mavsec: Securing the mavlink protocol for ardupilot/px4 unmanned aerial systems. *2019 15th International Wireless Communications & Mobile Computing Conference (IWCMC)*.

- Alqudsi, Y., & Makaraci, M. (2025). Uav swarms: Research, challenges, and future directions. *Journal of Engineering and Applied Science*.
- Dixit, B., A., A., Thahsin, A., Pathak, S., Kasbekar, G. S., & Maity, A. (2025). A novel cipher for enhancing mavlink security: Design, security analysis, and performance evaluation using a drone testbed.
- Du, F., Ge, J., Wang, W., Zou, Y., Chang, S.-Y., & Fan, W. (2024). Exploiting the vulnerabilities in mavlink protocol for uav hijacking. *2024 17th International Conference on Security of Information and Networks (SIN)*.
- Ficco, M., Palmiero, R., Rak, M., & Granata, D. (2022). Mavlink protocol for unmanned aerial vehicle: Vulnerabilities analysis. *2022 IEEE Intl Conf on Dependable, Autonomic and Secure Computing, Intl Conf on Pervasive Intelligence and Computing, Intl Conf on Cloud and Big Data Computing, Intl Conf on Cyber Science and Technology Congress (DASC/PiCom/CBDCCom/CyberSciTech)*.
- Friess, C., Niculescu, V., Polonelli, T., Magno, M., & Benini, L. (2024). Fully onboard slam for distributed mapping with a swarm of nano-drones. *IEEE Internet of Things Journal*.
- Koubâa, A., Allouch, A., Alajlan, M., Javed, Y., Belghith, A., & Khargui, M. (2019). Micro air vehicle link (mavlink) in a nutshell: A survey. *IEEE Access*.
- MAVLink Project. (2019). *Mavlink guide: Mavlink versions*. https://mavlink.io/en/guide/mavlink_version.html
- Nakamura, M., Ishii, H., & Dibaji, S. M. (2021). Resiliency against malicious agents in maximum-based consensus. *SICE Journal of Control, Measurement, and System Integration*.
- Olfati-Saber, R., Fax, J. A., & Murray, R. M. (2007). Consensus and cooperation in networked multi-agent systems. *Proceedings of the IEEE*.
- Pasqualetti, F., Bicchi, A., & Bullo, F. (2012). Consensus computation in unreliable networks: A system theoretic approach. *IEEE Transactions on Automatic Control*.
- Saldaña, D., Prorok, A., Sundaram, S., Campos, M. F. M., & Kumar, V. (2017). Resilient consensus for time-varying networks of dynamic agents. *2017 American Control Conference (ACC)*.
- Stallings, W. (2011). *Network security essentials: Applications and standards* (4th ed.). Pearson.
- Stolfi, D. H., & Danoy, G. (2025). Escorting drone swarm formation: A swarm intelligence and evolutionary optimisation approach. *Swarm Intelligence*.